



SpringMVC 第二天

框架课程

1. 课前回顾

1、Springmvc 介绍？ Springmvc 是 Spring 公司

2、Springmvc 入门程序

第一步：Web 工程

第二步：导 Jar 包

第三步：web.xml 配置前端控制器 servlet Filter

*.do.action /拦截所有不包含 jsp /*拦截所有（真拦截）

第四步：配置上下文 springmvc.xml 配置扫描@Controller

第五步：Handler Controller 程序员自己写的

@RequestMapping(value="/item/queryItem.action")

Public ModelAndView queryItem(){

 跳转/WEB-INF/jsp/itemList.jsp
}

商品列表查询

3、springmvc 原理图



用户请求到前端控制器、让处理器映射器去找相应的路径 对应的方法

返回找到的方法

前端控制器、让处理器适配器 去执行此方法（执行前绑定参数）

返回 ModelAndView

前端控制器、让视图解析器 数据填充在.jsp 的标签处、html

中心

三大组件

写的

Handler JSP

4、参数绑定 简单类型 queryItem(Integer id) jsp 页面上 input type=text
name=id

POJO Item

修改商品提交 input type=text name = pojo 里面要一致

包装 POJO QueryVo（里面 Item） name=item.id

自定义参数绑定



日期类型

Yyyy-MM_dd

2. 课程计划

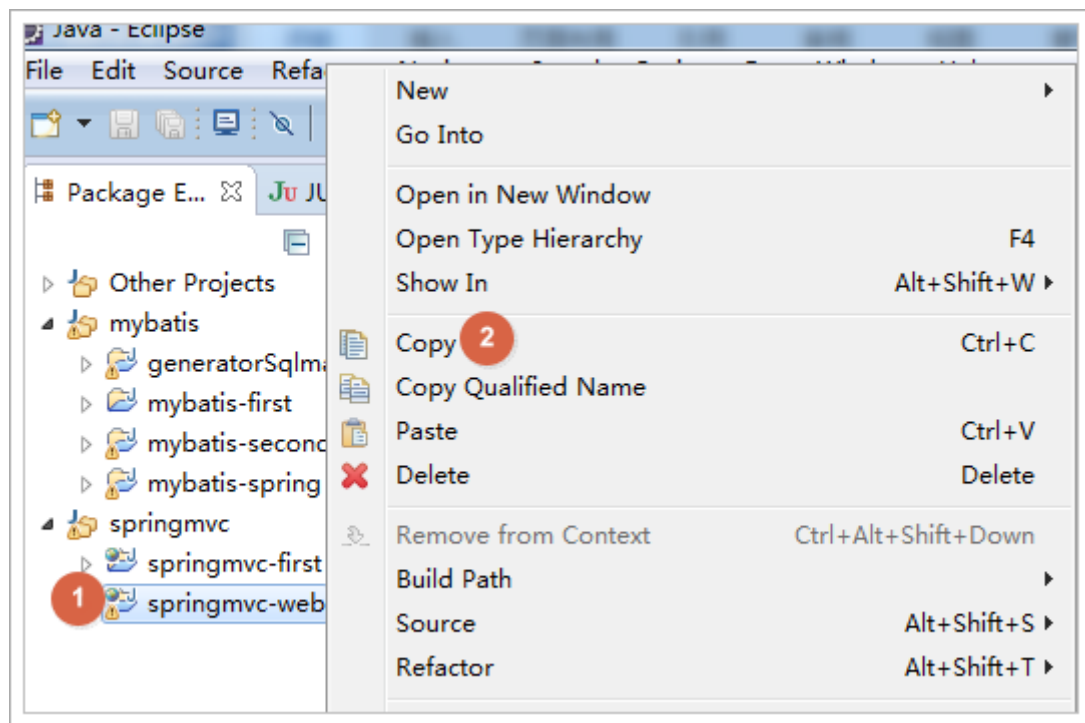
- 1、高级参数绑定
 - a) 数组类型的参数绑定
 - b) List 类型的绑定
- 2、@RequestMapping 注解的使用
- 3、Controller 方法返回值
- 4、Springmvc 中异常处理
- 5、图片上传处理
- 6、Json 数据交互
- 7、Springmvc 实现 RESTful
- 8、拦截器

3. 高级参数绑定

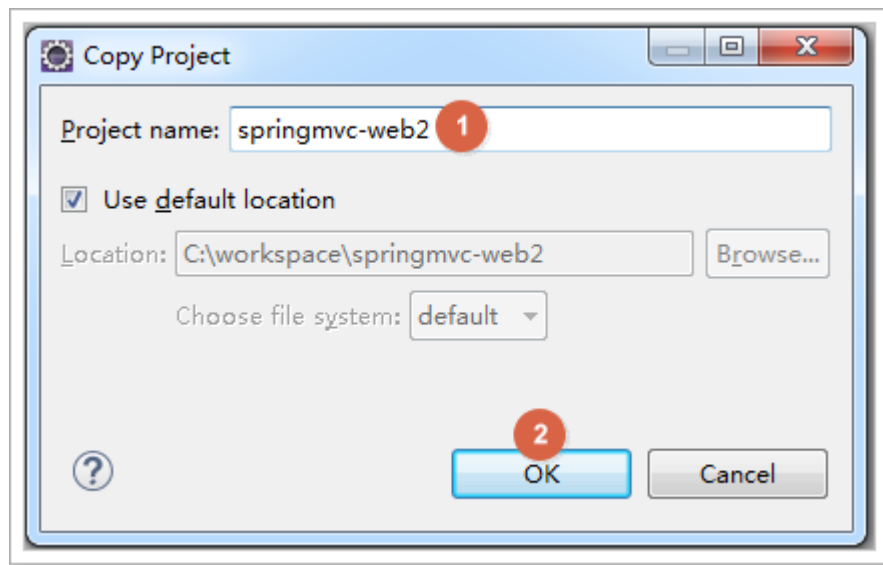
3.1. 复制工程

把昨天的 springmvc-web 工程复制一份，作为今天开发的工程

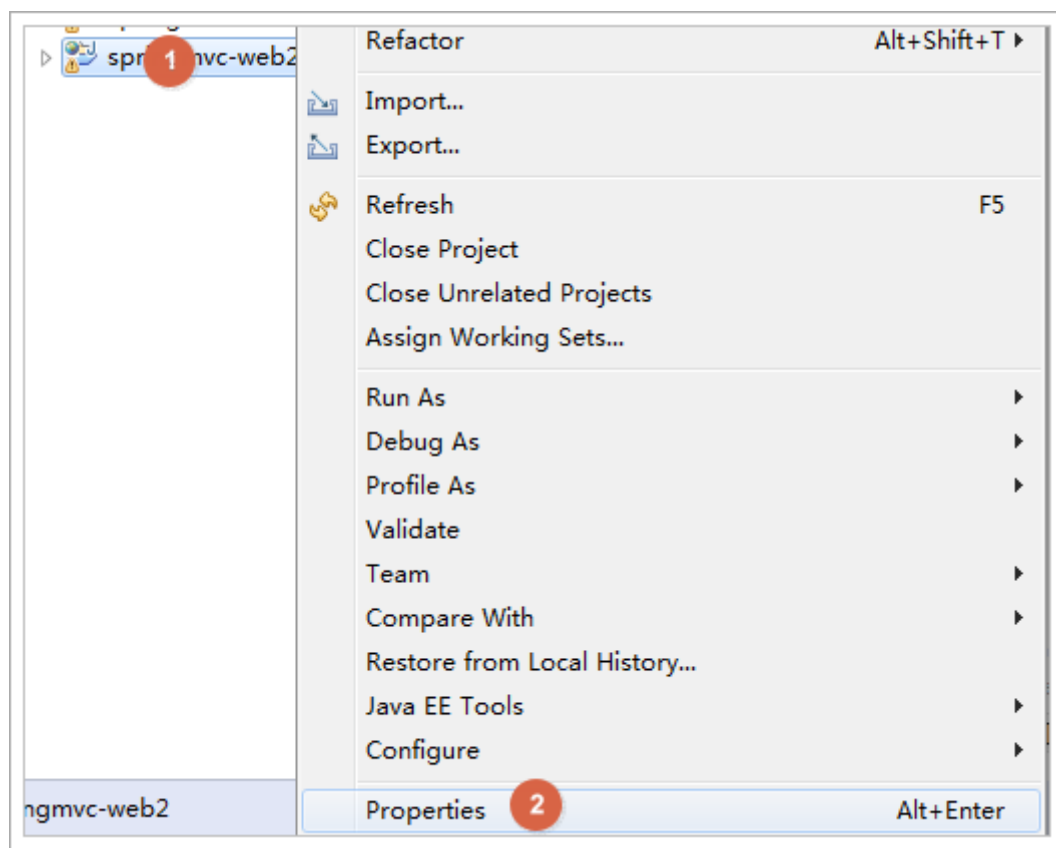
复制工程，如下图：



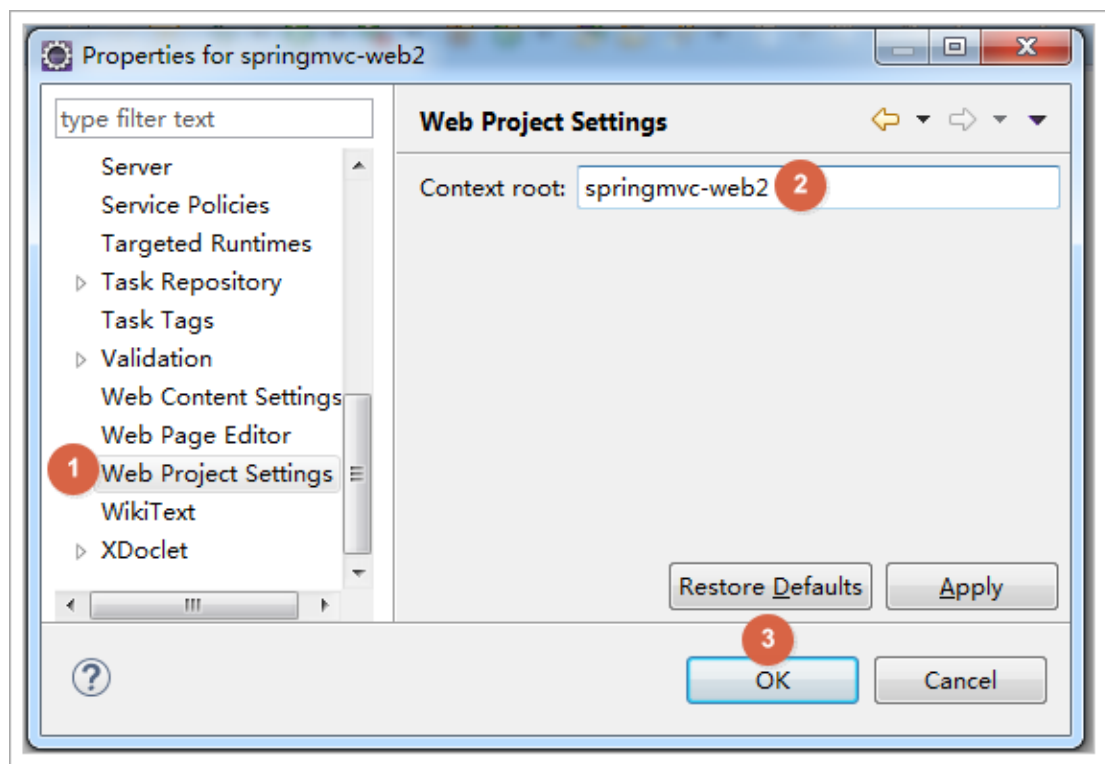
粘贴并修改工程名为 web2，如下图：



工程右键点击，如下图：



修改工程名，如下图：



3.2. 绑定数组

3.2.1. 需求

在商品列表页面选中多个商品，然后删除。

3.2.2. 需求分析

功能要求商品列表页面中的每个商品前有一个 checkbox，选中多个商品后点击删除按钮把商品 id 传递给 Controller，根据商品 id 删除商品信息。

我们演示可以获取 id 的数组即可

3.2.3. Jsp 修改

修改 itemList.jsp 页面,增加多选框，提交 url 是 queryItem.action

```
<form action="${pageContext.request.contextPath }/queryItem.action"
method="post">
```

查询条件:

```
<table width="100%" border=1>
<tr>
```



```
<td>商品 id<input type="text" name="item.id" /></td>
<td>商品名称<input type="text" name="item.name" /></td>
<td><input type="submit" value="查询"/></td>
</tr>
</table>
商品列表:
<table width="100%" border=1>
<tr>
<td>选择</td>
<td>商品名称</td>
<td>商品价格</td>
<td>生产日期</td>
<td>商品描述</td>
<td>操作</td>
</tr>
<c:forEach items="${itemList }" var="item">
<tr>
<td><input type="checkbox" name="ids" value="${item.id}"/></td>
<td>${item.name }</td>
<td>${item.price }</td>
<td><fmt:formatDate value="${item.createtime}" pattern="yyyy-MM-dd
HH:mm:ss"/></td>
<td>${item.detail }</td>

<td><a
href="${pageContext.request.contextPath }/itemEdit.action?id=${item.
id}">修改</a></td>

</tr>
</c:forEach>

</table>
</form>
```

页面选中多个 checkbox 向 controller 方法传递

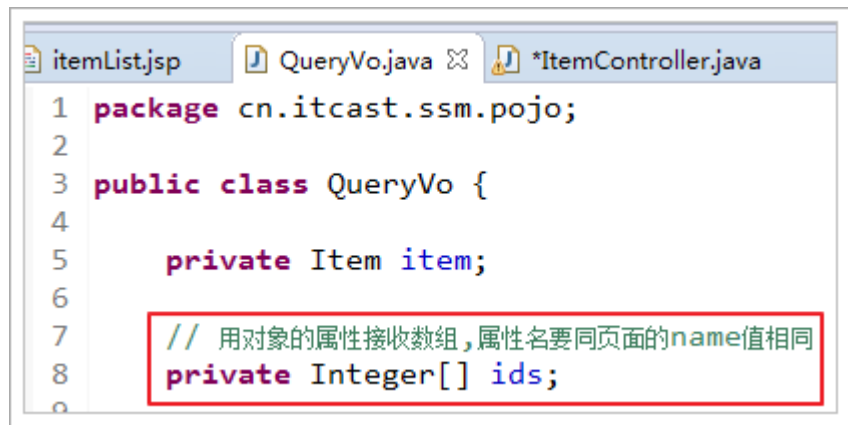
本身属于一个 form 表单，提交 url 是 queryItem.action



3.2.4. Controller

Controller 方法中可以用 String[]接收，或者 pojo 的 String[]属性接收。两种方式任选其一即可。

定义 QueryVo，如下图：



```
1 package cn.itcast.ssm.pojo;
2
3 public class QueryVo {
4
5     private Item item;
6
7     // 用对象的属性接收数组，属性名要同页面的name值相同
8     private Integer[] ids;
9 }
```

ItemController 修改 queryItem 方法：

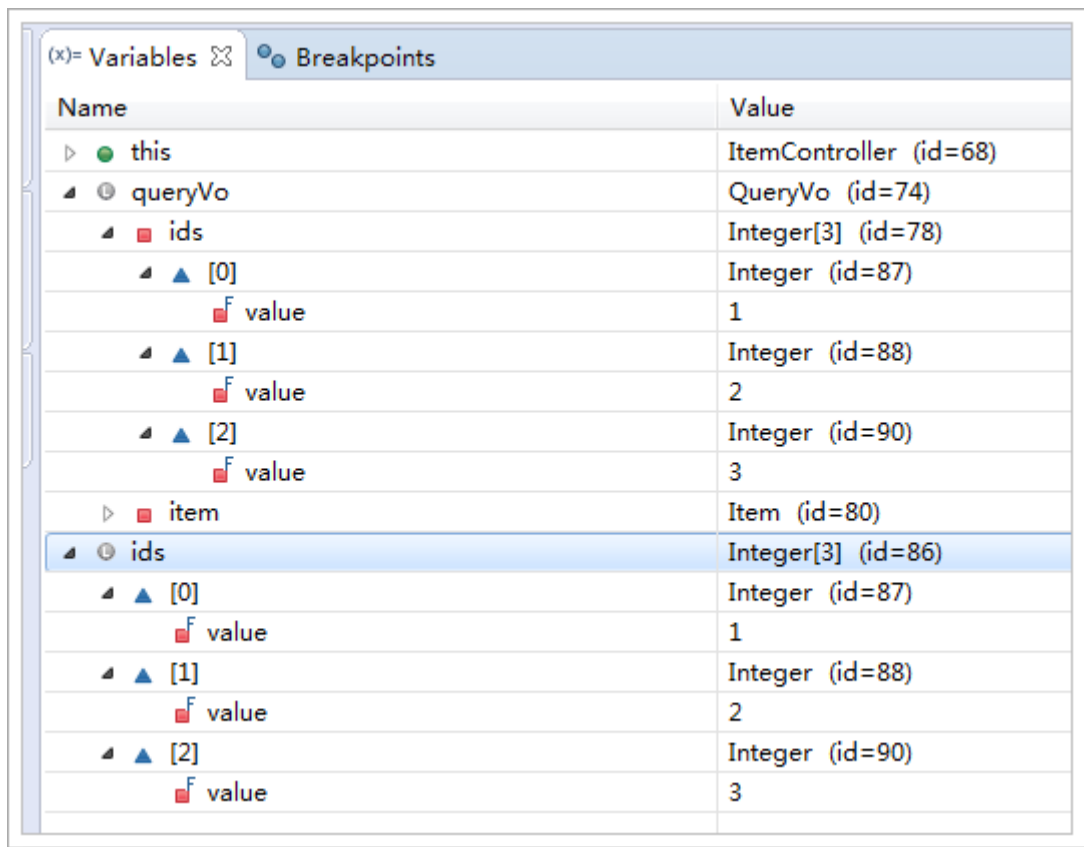
```
/**
 * 包装类型 绑定数组类型，可以使用两种方式，pojo 的属性接收，和直接接收
 *
 * @param queryVo
 * @return
 */
@RequestMapping("queryItem")
public String queryItem(QueryVo queryVo, Integer[] ids) {

    System.out.println(queryVo.getItem().getId());
    System.out.println(queryVo.getItem().getName());

    System.out.println(queryVo.getIds().length);
    System.out.println(ids.length);

    return "success";
}
```


效果，如下图：



Name	Value
this	ItemController (id=68)
queryVo	QueryVo (id=74)
ids	Integer[3] (id=78)
[0]	Integer (id=87)
value	1
[1]	Integer (id=88)
value	2
[2]	Integer (id=90)
value	3
item	Item (id=80)
ids	Integer[3] (id=86)
[0]	Integer (id=87)
value	1
[1]	Integer (id=88)
value	2
[2]	Integer (id=90)
value	3

3.3. 将表单的数据绑定到 List

3.3.1. 需求

实现商品数据的批量修改。

3.3.2. 开发分析

开发分析

1. 在商品列表页面中可以对商品信息进行修改。
2. 可以批量提交修改后的商品数据。

3.3.3. 定义 pojo

List 中存放对象，并将定义 List 放在包装类 QueryVo 中



使用包装 pojo 对象接收，如下图：

```
itemList.jsp  ItemController.java  QueryVo.java
5 public class QueryVo {
6
7     private Item item;
8
9     // 用对象的属性接收数组,属性名要同页面的name值相同
10    private Integer[] ids;
11
12    // 用对象的属性接收List集合
13    private List<Item> itemList;
```

3.3.4. Jsp 改造

前端页面应该显示的 html 代码，如下图：

```
<tr>
    <td><input type="checkbox" name="ids" value="1"/></td>
    <td><input type="text" name="itemList[0].name" value="台式机"/></td>
    <td><input type="text" name="itemList[0].price" value="6999.0"/></td>
    <td><input type="text" name="itemList[0].createtime" value="2016-12-13"/></td>
    <td><input type="text" name="itemList[0].detail" value="性价比很高!"/></td>
    <td><a href="/springmvc-web2/itemEdit.action?id=1">修改</a></td>
</tr>

<tr>
    <td><input type="checkbox" name="ids" value="2"/></td>
    <td><input type="text" name="itemList[1].name" value="笔记本"/></td>
    <td><input type="text" name="itemList[1].price" value="6000.0"/></td>
    <td><input type="text" name="itemList[1].createtime" value="2015-02-09"/></td>
    <td><input type="text" name="itemList[1].detail" value="笔记本性能好,"/></td>
    <td><a href="/springmvc-web2/itemEdit.action?id=2">修改</a></td>
</tr>

<tr>
    <td><input type="checkbox" name="ids" value="3"/></td>
    <td><input type="text" name="itemList[2].name" value="背包"/></td>
    <td><input type="text" name="itemList[2].price" value="200.0"/></td>
    <td><input type="text" name="itemList[2].createtime" value="2015-02-06"/></td>
    <td><input type="text" name="itemList[2].detail" value="名牌背包,容量大"/></td>
    <td><a href="/springmvc-web2/itemEdit.action?id=3">修改</a></td>
</tr>
```



分析发现：name 属性必须是 list 属性名+下标+元素属性。

Jsp 做如下改造：

```
<c:forEach items="${itemList }" var="item" varStatus="s">
<tr>
    <td><input type="checkbox" name="ids" value="${item.id}"/></td>
    <td>
        <input type="hidden" name="itemList[${s.index}].id"
value="${item.id }"/>
        <input type="text" name="itemList[${s.index}].name"
value="${item.name }"/>
    </td>
    <td><input type="text" name="itemList[${s.index}].price"
value="${item.price }"/></td>
    <td><input type="text" name="itemList[${s.index}].createtime"
value="<fmt:formatDate value="${item.createtime}" pattern="yyyy-MM-dd
HH:mm:ss"/>"/></td>
    <td><input type="text" name="itemList[${s.index}].detail"
value="${item.detail }"/></td>

    <td><a
href="${pageContext.request.contextPath }/itemEdit.action?id=${item.
id}">修改</a></td>

</tr>
</c:forEach>
```

`${current}` 当前这次迭代的（集合中的）项

`${status.first}` 判断当前项是否为集合中的第一项，返回值为 true 或 false

`${status.last}` 判断当前项是否为集合中的最

varStatus 属性常用参数总结下：

`${status.index}` 输出行号，从 0 开始。

`${status.count}` 输出行号，从 1 开始。

`${status.后一项}`，返回值为 true 或 false

begin、end、step 分别表示：起始序号，结束序号，跳跃步伐。

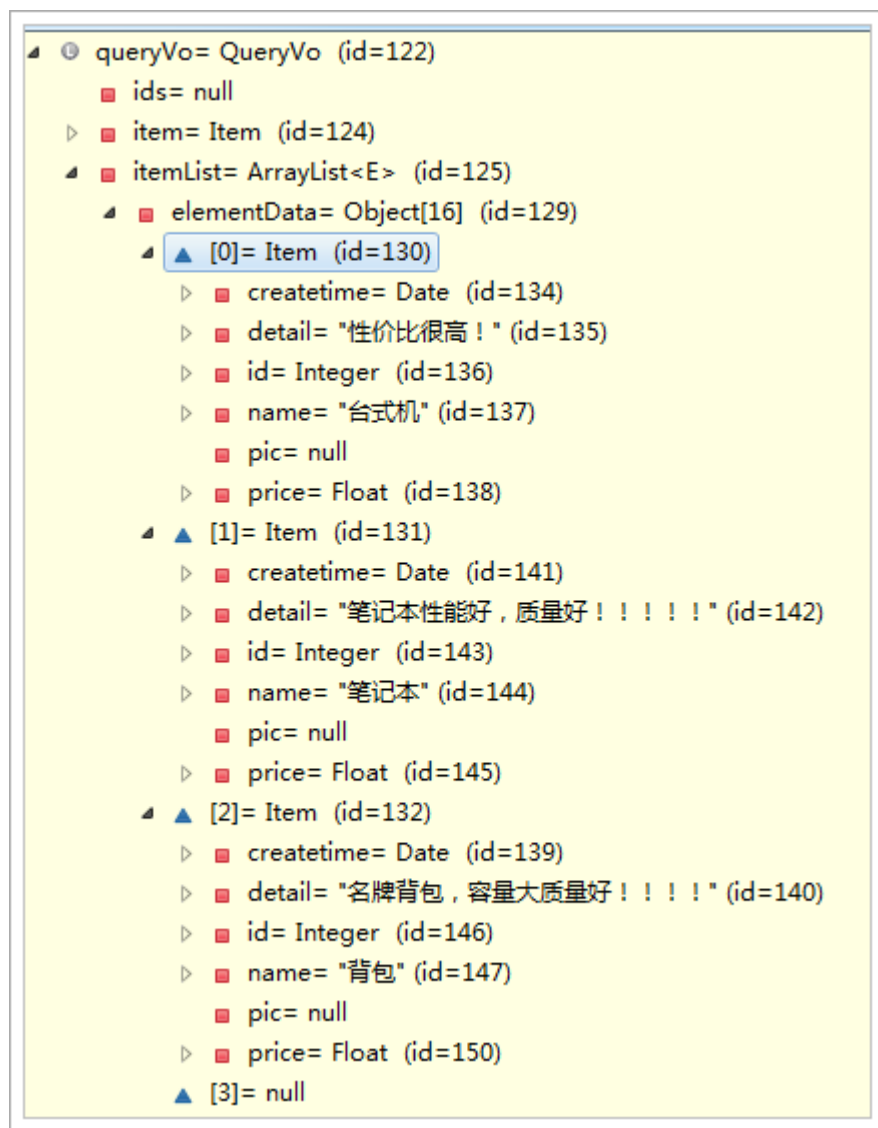
3.3.5. 效果

这里只演示 List 的绑定，能够接收到 list 数据。

可以拿到数据即可，不做数据库的操作。



测试效果如下图：



注意：接收 List 类型的数据必须是 pojo 的属性，如果方法的形参为 ArrayList 类型无法正确接收到数据。

4. @RequestMapping

通过 @RequestMapping 注解可以定义不同的处理器映射规则。

4.1. URL 路径映射

@RequestMapping(value="item")或@RequestMapping("/item")



value 的值是数组，可以将多个 url 映射到同一个方法

```
/**
 * 查询商品列表
 * @return
 */
@RequestMapping(value = { "itemList", "itemListAll" })
public ModelAndView queryItemList() {
    // 查询商品数据
    List<Item> list = this.itemService.queryItemList();

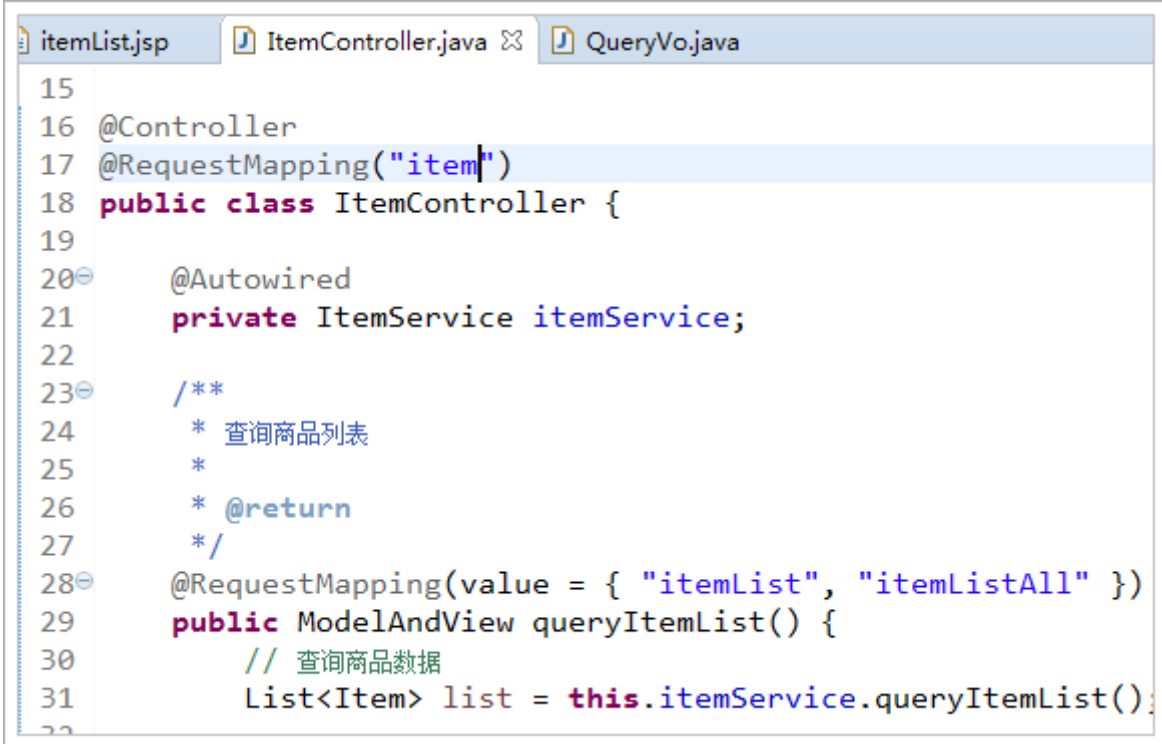
    // 创建 ModelAndView, 设置逻辑视图名
    ModelAndView mv = new ModelAndView("itemList");

    // 把商品数据放到模型中
    mv.addObject("itemList", list);
    return mv;
}
```

4.2. 添加在类上面

在 class 上添加@RequestMapping(url)指定通用请求前缀， 限制此类下的所有方法请求 url 必须以请求前缀开头

可以使用此方法对 url 进行分类管理，如下图：



```
itemList.jsp  ItemController.java  QueryVo.java
15
16 @Controller
17 @RequestMapping("item")
18 public class ItemController {
19
20     @Autowired
21     private ItemService itemService;
22
23     /**
24      * 查询商品列表
25      *
26      * @return
27      */
28     @RequestMapping(value = { "itemList", "itemListAll" })
29     public ModelAndView queryItemList() {
30         // 查询商品数据
31         List<Item> list = this.itemService.queryItemList();
32     }
33 }
```



此时需要进入 queryItem()方法的请求 url 为:

http://127.0.0.1:8080/springmvc-web2/item/itemList.action

或者

http://127.0.0.1:8080/springmvc-web2/item/itemListAll.action

注意: 学员练习此项后, 把类上的@RequestMapping 注释掉, 如下:

```
//@RequestMapping("item")  
public class ItemController {
```

以方便后面的练习

4.3. 请求方法限定

除了可以对 url 进行设置, 还可以限定请求进来的方法

◆ 限定 GET 方法

```
@RequestMapping(method = RequestMethod.GET)
```

如果通过 POST 访问则报错:

HTTP Status 405 - Request method 'POST' not supported

例如:

```
@RequestMapping(value = "itemList",method = RequestMethod.POST)
```

◆ 限定 POST 方法

```
@RequestMapping(method = RequestMethod.POST)
```

如果通过 GET 访问则报错:

HTTP Status 405 - Request method 'GET' not supported

◆ GET 和 POST 都可以

```
@RequestMapping(method = {RequestMethod.GET,RequestMethod.POST})
```

5. Controller 方法返回值

5.1. 返回 ModelAndView

controller 方法中定义 ModelAndView 对象并返回, 对象中可添加 model 数据、指定 view。

参考第一天的内容

5.2. 返回 void

在 Controller 方法形参上可以定义 request 和 response，使用 request 或 response 指定响应结果：

1、使用 request 转发页面，如下：

```
request.getRequestDispatcher("页面路径").forward(request, response);  
request.getRequestDispatcher("/WEB-INF/jsp/success.jsp").forward(request, response);
```

2、可以通过 response 页面重定向：

```
response.sendRedirect("url")  
response.sendRedirect("/springmvc-web2/itemEdit.action");
```

3、可以通过 response 指定响应结果，例如响应 json 数据如下：

```
response.getWriter().print("{\"abc\":123}");
```

5.2.1. 代码演示

以下代码一次测试，演示上面的效果

```
/**  
 * 返回 void 测试  
 *  
 * @param request  
 * @param response  
 * @throws Exception  
 */  
@RequestMapping("queryItem")  
public void queryItem(HttpServletRequest request, HttpServletResponse response) throws Exception {  
    // 1 使用 request 进行转发  
    //  
    request.getRequestDispatcher("/WEB-INF/jsp/success.jsp").forward(request,  
        // response);  
  
    // 2 使用 response 进行重定向到编辑页面  
    // response.sendRedirect("/springmvc-web2/itemEdit.action");
```



```
// 3 使用 response 直接显示
response.getWriter().print("{\"abc\":\"123\"}");
}
```

5.3. 返回字符串

5.3.1. 逻辑视图名

controller 方法返回字符串可以指定逻辑视图名，通过视图解析器解析为物理视图地址。

```
//指定逻辑视图名，经过视图解析器解析为jsp物理路径：
/WEB-INF/jsp/itemList.jsp
return "itemList";
```

参考第一天内容

5.3.2. Redirect 重定向

Controller 方法返回字符串可以重定向到一个 url 地址
如下商品修改提交后重定向到商品编辑页面。

```
/**
 * 更新商品
 *
 * @param item
 * @return
 */
@RequestMapping("updateItem")
public String updateItemById(Item item) {
    // 更新商品
    this.itemService.updateItemById(item);

    // 修改商品成功后，重定向到商品编辑页面
    // 重定向后浏览器地址栏变更为重定向的地址，
    // 重定向相当于执行了新的 request 和 response，所以之前的请求参数都会丢失
    // 如果要指定请求参数，需要在重定向的 url 后面添加 ?itemId=1 这样的请求参数
    return "redirect:/itemEdit.action?itemId=" + item.getId();
}
```

5.3.3. forward 转发

Controller 方法执行后继续执行另一个 Controller 方法

如下商品修改提交后转向到商品修改页面，修改商品的 id 参数可以带到商品修



改方法中。

```
/**
 * 更新商品
 *
 * @param item
 * @return
 */
@RequestMapping("updateItem")
public String updateItemById(Item item) {
    // 更新商品
    this.itemService.updateItemById(item);

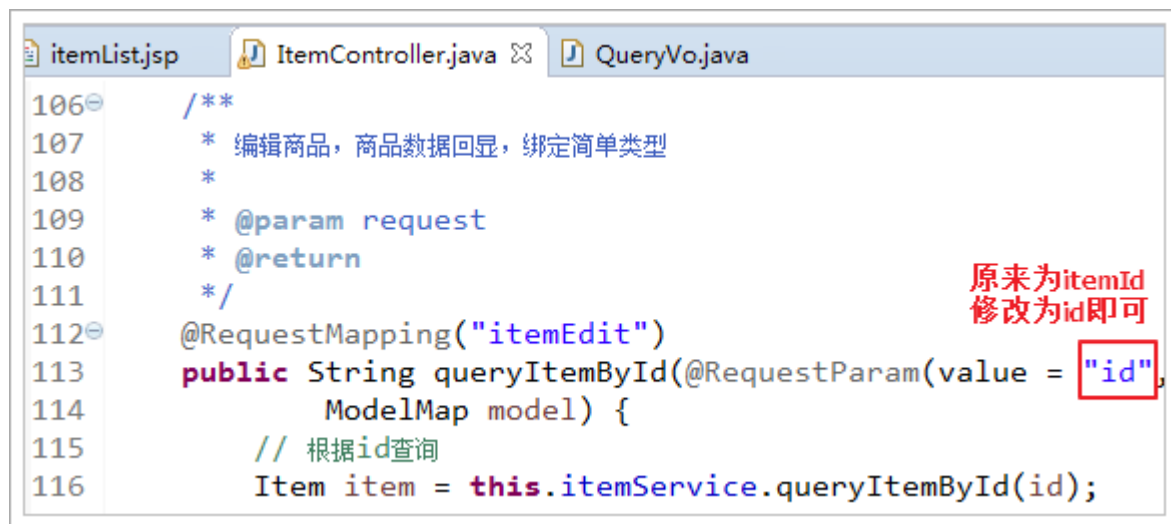
    // 修改商品成功后，重定向到商品编辑页面
    // 重定向后浏览器地址栏变更为重定向的地址，
    // 重定向相当于执行了新的 request 和 response，所以之前的请求参数都会丢失
    // 如果要指定请求参数，需要在重定向的 url 后面添加 ?itemId=1 这样的请求参数
    // return "redirect:/itemEdit.action?itemId=" + item.getId();

    // 修改商品成功后，继续执行另一个方法
    // 使用转发的方式实现。转发后浏览器地址栏还是原来的请求地址，
    // 转发并没有执行新的 request 和 response，所以之前的请求参数都存在
    return "forward:/itemEdit.action";
}
//结果转发到editItem.action，request可以带过去
return "forward: /itemEdit.action";
```

需要修改之前编写的根据 id 查询商品方法

因为请求进行修改商品时，请求参数里面只有 id 属性，没有 itemId 属性

修改，如下图：



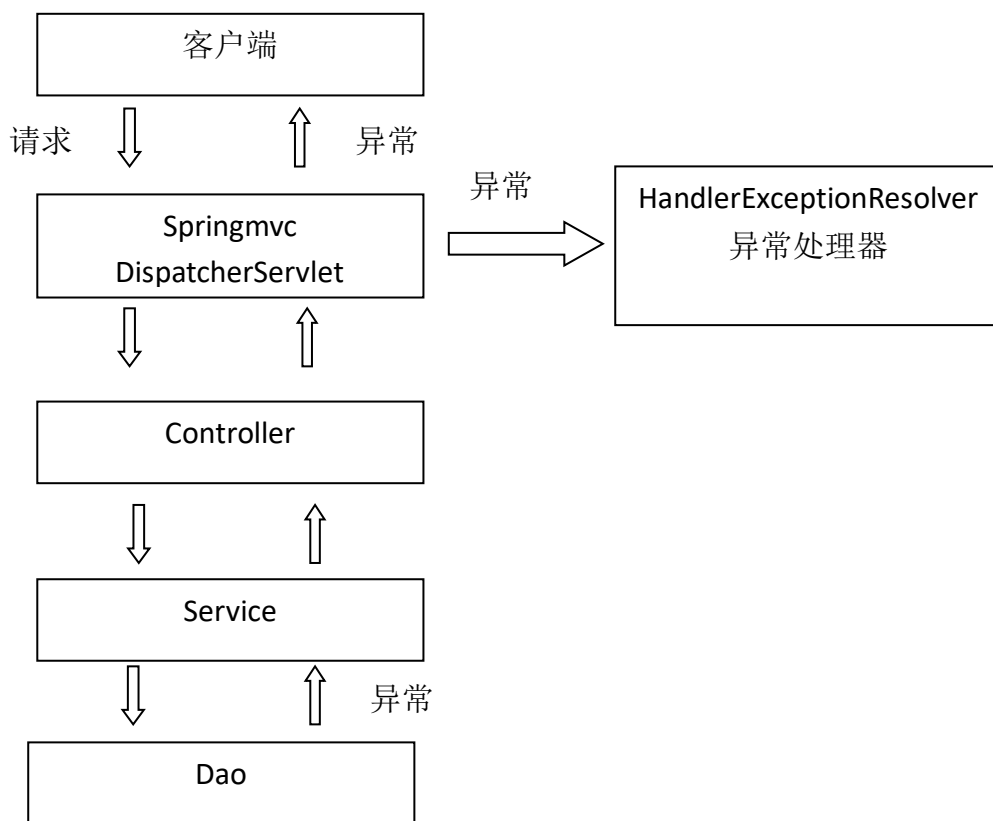
6. 异常处理器

springmvc 在处理请求过程中出现异常信息交由异常处理器进行处理，自定义异常处理器可以实现一个系统的异常处理逻辑。

6.1. 异常处理思路

系统中异常包括两类：预期异常和运行时异常 `RuntimeException`，前者通过捕获异常从而获取异常信息，后者主要通过规范代码开发、测试通过手段减少运行时异常的发生。

系统的 dao、service、controller 出现都通过 throws Exception 向上抛出，最后由 springmvc 前端控制器交由异常处理器进行异常处理，如下图：





6.2. 自定义异常类

为了区别不同的异常,通常根据异常类型进行区分,这里我们创建一个自定义系统异常。

如果 controller、service、dao 抛出此类异常说明是系统预期处理的异常信息。

```
public class MyException extends Exception {  
    // 异常信息  
    private String message;  
  
    public MyException() {  
        super();  
    }  
  
    public MyException(String message) {  
        super();  
        this.message = message;  
    }  
  
    public String getMessage() {  
        return message;  
    }  
  
    public void setMessage(String message) {  
        this.message = message;  
    }  
}
```

6.3. 自定义异常处理器

```
public class CustomHandleException implements HandlerExceptionResolver {  
  
    @Override  
    public ModelAndView resolveException(HttpServletRequest request,  
        HttpServletResponse response, Object handler,  
        Exception exception) {  
        // 定义异常信息  
        String msg;  
  
        // 判断异常类型
```



```
if (exception instanceof MyException) {
    // 如果是自定义异常，读取异常信息
    msg = exception.getMessage();
} else {
    // 如果是运行时异常，则取错误堆栈，从堆栈中获取异常信息
    Writer out = new StringWriter();
    PrintWriter s = new PrintWriter(out);
    exception.printStackTrace(s);
    msg = out.toString();
}

// 把错误信息发给相关人员, 邮件, 短信等方式
// TODO

// 返回错误页面，给用户友好页面显示错误信息
ModelAndView modelAndView = new ModelAndView();
modelAndView.addObject("msg", msg);
modelAndView.setViewName("error");

return modelAndView;
}
```

6.4. 异常处理器配置

在 springmvc.xml 中添加：

```
<!-- 配置全局异常处理器 -->
<bean
    id="customHandleException"
    class="cn.itcast.ssm.exception.CustomHandleException"/>
```

6.5. 错误页面

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
    "http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
```



```
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>Insert title here</title>
</head>
<body>

    <h1>系统发生异常了! </h1>
    <br />
    <h1>异常信息</h1>
    <br />
    <h2>${msg }</h2>

</body>
</html>
```

6.6. 异常测试

修改 ItemController 方法 “queryItemList”，抛出异常：

```
/**
 * 查询商品列表
 *
 * @return
 * @throws Exception
 */
@RequestMapping(value = { "itemList", "itemListAll" })
public ModelAndView queryItemList() throws Exception {
    // 自定义异常
    if (true) {
        throw new MyException("自定义异常出现了~");
    }

    // 运行时异常
    int a = 1 / 0;

    // 查询商品数据
    List<Item> list = this.itemService.queryItemList();
    // 创建 ModelAndView, 设置逻辑视图名
    ModelAndView mv = new ModelAndView("itemList");
    // 把商品数据放到模型中
    mv.addObject("itemList", list);

    return mv;
}
```

7. 上传图片

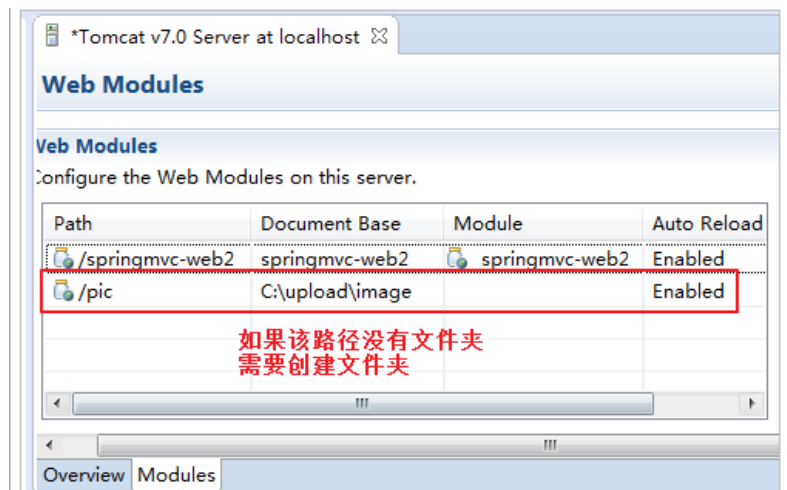
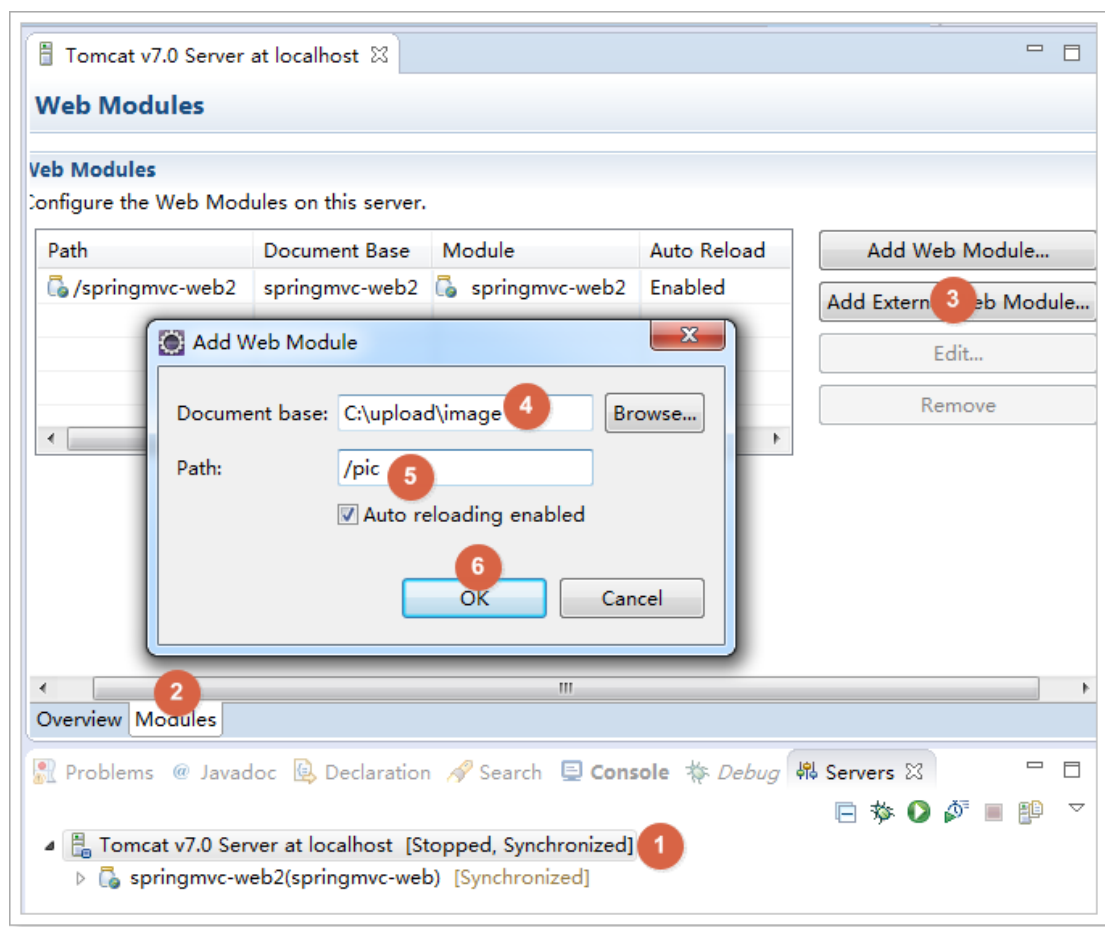
7.1. 配置虚拟目录

在 tomcat 上配置图片虚拟目录，在 tomcat 下 conf/server.xml 中添加：

```
<Context docBase="D:\develop\upload\temp" path="/pic" reloadable="false"/>
```

访问 <http://localhost:8080/pic> 即可访问 D:\develop\upload\temp 下的图片。

也可以通过 eclipse 配置，如下图：

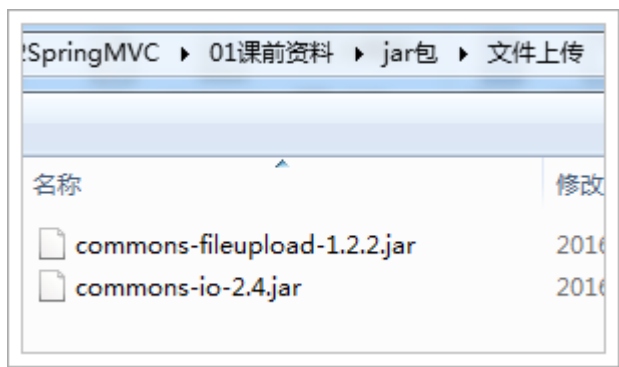


复制一张图片到存放图片的文件夹，使用浏览器访问测试效果，如下图：



7.2. 加入 jar 包

实现图片上传需要加入的 jar 包，如下图：



把两个 jar 包放到工程的 lib 文件夹中

7.3. 配置上传解析器

在 springmvc.xml 中配置文件上传解析器



```
<!-- 文件上传,id 必须设置为 multipartResolver -->
<bean id="multipartResolver"
      class="org.springframework.web.multipart.commons.CommonsMultipart
      Resolver">
  <!-- 设置文件上传大小 -->
  <property name="maxUploadSize" value="5000000" />
</bean>
```

7.4. jsp 页面修改

在商品修改页面，打开图片上传功能，如下图：

```
31 <tr>
32   <td>商品图片</td>
33   <td>
34     <c:if test="${item.pic != null}">
35       
36       <br/>
37     </c:if>
38     <input type="file" name="pictureFile"/>
39   </td>
40 </tr>
```

设置表单可以进行文件上传，如下图：

```
10
11 </head>
12 <body>
13   <!-- 上传图片是需要指定属性 enctype="multipart/form-data" -->
14   <!-- <form id="itemForm" action="" method="post" enc -->
15   <form id="itemForm" action="${pageContext.request.co -->
16     enctype="multipart/form-data">
17     <input type="hidden" name="id" value="${item.id -->
18     <table width="100%" border=1>
19       <tr>
20         <td>商品名称</td>
```

7.5. 图片上传

在更新商品方法中添加图片上传逻辑

```
/**
 * 更新商品
 *
```




```
* @param item
* @return
* @throws Exception
*/
@RequestMapping("updateItem")
public String updateItemById(Item item, MultipartFile pictureFile)
throws Exception {
    // 图片上传
    // 设置图片名称，不能重复，可以使用 uuid
    String picName = UUID.randomUUID().toString();

    // 获取文件名
    String oriName = pictureFile.getOriginalFilename();
    // 获取图片后缀
    String extName = oriName.substring(oriName.lastIndexOf("."));

    // 开始上传
    pictureFile.transferTo(new File("C:/upload/image/" + picName +
extName));

    // 设置图片名到商品中
    item.setPic(picName + extName);
    // -----
    // 更新商品
    this.itemService.updateItemById(item);

    return "forward:/itemEdit.action";
}
```

效果，如下图：

商品名称	台式机33333312
商品价格	6999.0
商品生产日期	2016-12-13 13:22:53
商品图片	 选择文件 未选择任何文件
商品简介	性价比很高！
提交	



8. json 数据交互

8.1. @RequestBody

作用：

@RequestBody 注解用于读取 http 请求的内容(字符串)，通过 springmvc 提供的 `HttpMessageConverter` 接口将读到的内容（json 数据）转换为 java 对象并绑定到 Controller 方法的参数上。

传统的请求参数：

`itemEdit.action?id=1&name=zhangsan&age=12`

现在的请求参数：

使用 POST 请求，在请求体里面加入 json 数据

```
{
  "id": 1,
  "name": "测试商品",
  "price": 99.9,
  "detail": "测试商品描述",
  "pic": "123456.jpg"
}
```

本例子应用：

@RequestBody 注解实现接收 http 请求的 json 数据，将 json 数据转换为 java 对象进行绑定

8.2. @ResponseBody

作用：

@ResponseBody 注解用于将 Controller 的方法返回的对象，通过 springmvc 提供的 `HttpMessageConverter` 接口转换为指定格式的数据如：json,xml 等，通过 Response 响应给客户端

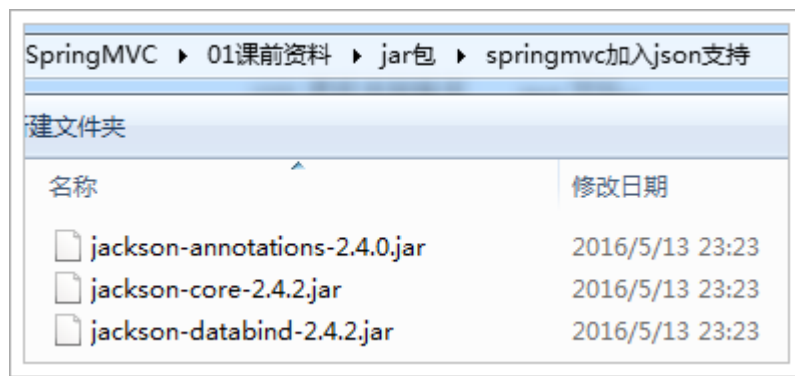
本例子应用：

@ResponseBody 注解实现将 Controller 方法返回 java 对象转换为 json 响应给客户端。

8.3. 请求 json，响应 json 实现：

8.3.1. 加入 jar 包

如果需要 springMVC 支持 json，必须加入 json 的处理 jar
我们使用 Jackson 这个 jar，如下图：

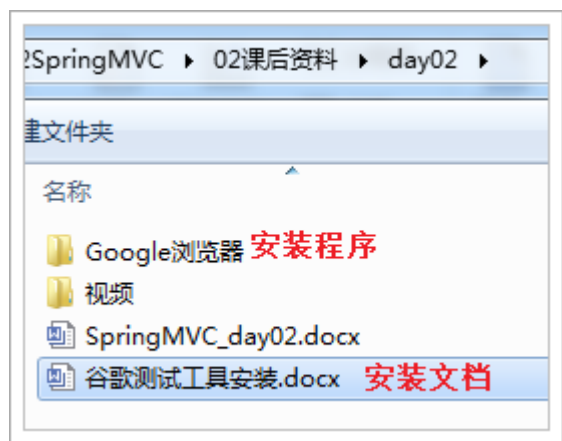


8.3.2. ItemController 编写

```
/**
 * 测试 json 的交互
 * @param item
 * @return
 */
@RequestMapping("testJson")
// @ResponseBody
public @ResponseBody Item testJson(@RequestBody Item item) {
    return item;
}
```

8.3.3. 安装谷歌浏览器测试工具

安装程序在课后资料
参考安装文档，如下图：





8.3.4. 测试方法

测试方法，如下图：

The screenshot shows a REST client interface with the following components and annotations:

- 1 输入测试url**: The URL input field contains `http://127.0.0.1:8080/springmvc-web2/jsonTest.action`.
- 2**: The HTTP method dropdown is set to `POST`.
- 3 输入测试数据**: The payload input field contains a JSON object:

```
{
  "id": 1,
  "name": "测试商品",
  "price": 99.9,
  "detail": "测试商品描述",
  "pic": "123456.jpg"
}
```
- 4**: The "Content-Type" header dropdown menu is open, showing `application/json` selected.
- 5**: The `Send` button is highlighted.

Other visible elements include tabs for `Raw`, `Form`, and `Headers` in the top section, and `Raw`, `Form`, `Files (0)`, and `Payload` in the bottom section. The bottom right shows a response preview with a status of `200` and a content type of `application/json`.

8.3.5. 测试结果

如下图：

Status: 200 OK Loading time: 9945 ms

Request headers: Origin: chrome-extension://nihlmfmfbogaganlnkbeenfcjgapclbb
User-Agent: Mozilla/5.0 (Windows NT 6.1; Win64; x64) AppleWebKit/537.36 (KHTML like Gecko) Chrome/51.0.2704.84 Safari/537.36
Content-Type: application/json 请求的是json
Accept: */*
Accept-Encoding: gzip, deflate
Accept-Language: zh-CN,zh;q=0.8,zh-TW;q=0.6,en;q=0.4,ja;q=0.2
Cookie: JSESSIONID=FCB2FED8C596AA80C5916F641E944312

Response headers: Server: Apache-Coyote/1.1
Content-Type: application/json; charset=UTF-8 响应的也是json
Transfer-Encoding: chunked
Date: Sat, 30 Jul 2016 22:41:51 GMT

Raw JSON Response

Copy to clipboard Save as file

```
{
  "id": 1,
  "name": "测试商品",
  "price": 99.9,
  "detail": "测试商品描述",
  "pic": "123456.jpg",
  "createtime": null
}
```

8.3.6. 配置 json 转换器

如果不使用注解驱动<mvc:annotation-driven />，就需要给处理器适配器配置 json 转换器，参考之前学习的自定义参数绑定。

在 springmvc.xml 配置文件中，给处理器适配器加入 json 转换器：

```
<!--处理器适配器 -->
<bean
class="org.springframework.web.servlet.mvc.method.annotation.R
equestMappingHandlerAdapter">
    <property name="messageConverters">
        <list>
            <bean
class="org.springframework.http.converter.json.MappingJacksonH
ttpMessageConverter"></bean>
```

```
</list>
</property>
</bean>
```

9. RESTful 支持

9.1. 什么是 restful?

Restful 就是一个资源定位及资源操作的风格。不是标准也不是协议，只是一种风格。基于这个风格设计的软件可以更简洁，更有层次，更易于实现缓存等机制。

资源：互联网所有的事物都可以被抽象为资源

资源操作：使用 POST、DELETE、PUT、GET，使用不同方法对资源进行操作。

分别对应 添加、 删除、修改、查询。

传统方式操作资源

http://127.0.0.1/item/queryItem.action?id=1	查询,GET
http://127.0.0.1/item/saveItem.action	新增,POST
http://127.0.0.1/item/updateItem.action	更新,POST
http://127.0.0.1/item/deleteItem.action?id=1	删除,GET 或 POST

使用 RESTful 操作资源

http://127.0.0.1/item/1	查询,GET
http://127.0.0.1/item	新增,POST
http://127.0.0.1/item	更新,PUT
http://127.0.0.1/item/1	删除,DELETE

9.2. 需求

RESTful 方式实现商品信息查询，返回 json 数据

9.3. 从 URL 上获取参数

使用 RESTful 风格开发的接口，根据 id 查询商品，接口地址是：

http://127.0.0.1/item/1



我们需要从 url 上获取商品 id，步骤如下：

1. 使用注解@RequestMapping("item/{id}")声明请求的 url
{xxx}叫做占位符，请求的 URL 可以是“item /1”或“item/2”
2. 使用(@PathVariable() Integer id)获取 url 上的数据

```
/**
 * 使用 RESTful 风格开发接口，实现根据 id 查询商品
 *
 * @param id
 * @return
 */
@RequestMapping("item/{id}")
@ResponseBody
public Item queryItemById(@PathVariable() Integer id) {
    Item item = this.itemService.queryItemById(id);
    return item;
}
```

如果@RequestMapping 中表示为"item/{id}"，id 和形参名称一致，@PathVariable 不用指定名称。如果不一致，例如"item/{ItemId}"则需要指定名称@PathVariable("itemId")。

http://127.0.0.1/item/123?id=1

注意两个区别

1. @PathVariable 是获取 url 上数据的。@RequestParam 获取请求参数的（包括 post 表单提交）
2. 如果加上@ResponseBody 注解，就不会走视图解析器，不会返回页面，目前返回的 json 数据。如果不加，就走视图解析器，返回页面

10. 拦截器

10.1. 定义

Spring Web MVC 的处理器拦截器类似于Servlet 开发中的过滤器Filter，用于对处理器进行预处理和后处理。



10.2. 拦截器定义

实现 HandlerInterceptor 接口，如下：

```
public class HandlerInterceptor1 implements HandlerInterceptor {
    // controller 执行后且视图返回后调用此方法
    // 这里可得到执行 controller 时的异常信息
    // 这里可记录操作日志
    @Override
    public void afterCompletion(HttpServletRequest arg0,
        HttpServletResponse arg1, Object arg2, Exception arg3)
        throws Exception {
        System.out.println("HandlerInterceptor1....afterCompletion");
    }

    // controller 执行后但未返回视图前调用此方法
    // 这里可在返回用户前对模型数据进行加工处理，比如这里加入公用信息以便页面显示
    @Override
    public void postHandle(HttpServletRequest arg0, HttpServletResponse
        arg1, Object arg2, ModelAndView arg3)
        throws Exception {
        System.out.println("HandlerInterceptor1....postHandle");
    }

    // Controller 执行前调用此方法
    // 返回 true 表示继续执行，返回 false 中止执行
    // 这里可以加入登录校验、权限拦截等
    @Override
    public boolean preHandle(HttpServletRequest arg0,
        HttpServletResponse arg1, Object arg2) throws Exception {
        System.out.println("HandlerInterceptor1....preHandle");
        // 设置为 true，测试使用
        return true;
    }
}
```

10.3. 拦截器配置

上面定义的拦截器再复制一份 HandlerInterceptor2，注意新的拦截器修改代码：

```
System.out.println("HandlerInterceptor2....preHandle");
```

在 springmvc.xml 中配置拦截器



```
<!-- 配置拦截器 -->
<mvc:interceptors>
    <mvc:interceptor>
        <!-- 所有的请求都进入拦截器 -->
        <mvc:mapping path="/*" />
        <!-- 配置具体的拦截器 -->
        <bean class="cn.itcast.ssm.interceptor.HandlerInterceptor1" />
    </mvc:interceptor>
    <mvc:interceptor>
        <!-- 所有的请求都进入拦截器 -->
        <mvc:mapping path="/*" />
        <!-- 配置具体的拦截器 -->
        <bean class="cn.itcast.ssm.interceptor.HandlerInterceptor2" />
    </mvc:interceptor>
</mvc:interceptors>
```

10.4. 正常流程测试

浏览器访问地址

<http://127.0.0.1:8080/springmvc-web2/itemList.action>

10.4.1. 运行流程

控制台打印：

HandlerInterceptor1..preHandle..

HandlerInterceptor2..preHandle..

HandlerInterceptor2..postHandle..

HandlerInterceptor1..postHandle..

HandlerInterceptor2..afterCompletion..

HandlerInterceptor1..afterCompletion..

10.5. 中断流程测试

浏览器访问地址

<http://127.0.0.1:8080/springmvc-web2/itemList.action>

10.5.1. 运行流程

HandlerInterceptor1 的 preHandler 方法返回 false，HandlerInterceptor2 返回 true，

运行流程如下：

HandlerInterceptor1..preHandle..

从日志看出第一个拦截器的 `preHandler` 方法返回 `false` 后第一个拦截器只执行了 `preHandler` 方法，其它两个方法没有执行，第二个拦截器的所有方法不执行，且 `Controller` 也不执行了。

HandlerInterceptor1 的 `preHandler` 方法返回 `true`，HandlerInterceptor2 返回 `false`，运行流程如下：

HandlerInterceptor1..preHandle..

HandlerInterceptor2..preHandle..

HandlerInterceptor1..afterCompletion..

从日志看出第二个拦截器的 `preHandler` 方法返回 `false` 后第一个拦截器的 `postHandler` 没有执行，第二个拦截器的 `postHandler` 和 `afterCompletion` 没有执行，且 `controller` 也不执行了。

总结：

`preHandle` 按拦截器定义顺序调用

`postHandler` 按拦截器定义逆序调用

`afterCompletion` 按拦截器定义逆序调用

`postHandler` 在拦截器链内所有拦截器返回成功调用

`afterCompletion` 只有 `preHandle` 返回 `true` 才调用

10.6. 拦截器应用

10.6.1. 处理流程

- 1、有一个登录页面，需要写一个 `Controller` 访问登录页面
- 2、登录页面有一提交表单的动作。需要在 `Controller` 中处理。
 - a) 判断用户名密码是否正确（在控制台打印）
 - b) 如果正确,向 `session` 中写入用户信息（写入用户名 `username`）
 - c) 跳转到商品列表
- 3、拦截器。
 - a) 拦截用户请求，判断用户是否登录（登录请求不能拦截）



- b) 如果用户已经登录。放行
- c) 如果用户未登录，跳转到登录页面。

10.6.2. 编写登录 jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>Insert title here</title>
</head>
<body>

<form action="${pageContext.request.contextPath }/user/login.action">
<label>用户名: </label>
<br>
<input type="text" name="username">
<br>
<label>密码: </label>
<br>
<input type="password" name="password">
<br>
<input type="submit">

</form>

</body>
</html>
```

10.6.3. 用户登陆 Controller

```
@Controller
@RequestMapping("user")
public class UserController {

    /**
     * 跳转到登录页面
     *
     * @return
     */
    @RequestMapping("toLogin")
    public String toLogin() {
```



```
        return "login";
    }

    /**
     * 用户登录
     *
     * @param username
     * @param password
     * @param session
     * @return
     */
    @RequestMapping("login")
    public String login(String username, String password, HttpSession session) {
        // 校验用户登录
        System.out.println(username);
        System.out.println(password);

        // 把用户名放到 session 中
        session.setAttribute("username", username);

        return "redirect:/item/itemList.action";
    }
}
```

10.6.4. 编写拦截器

```
@Override
public boolean preHandle(HttpServletRequest request,
    HttpServletResponse response, Object arg2) throws Exception {
    // 从 request 中获取 session
    HttpSession session = request.getSession();
    // 从 session 中获取 username
    Object username = session.getAttribute("username");
    // 判断 username 是否为 null
    if (username != null) {
        // 如果不为空则放行
        return true;
    } else {
```

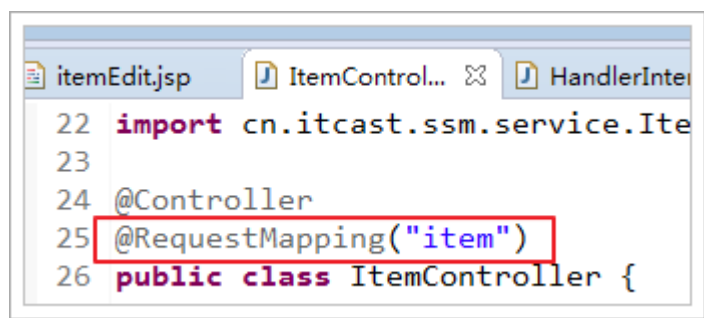


```
// 如果为空则跳转到登录页面
response.sendRedirect(request.getContextPath() +
"/user/toLogin.action");
}

return false;
}
```

10.6.5. 配置拦截器

只能拦截商品的 url，所以需要修改 ItemController，让所有的请求都必须以 item 开头，如下图：



在 springmvc.xml 配置拦截器

```
<mvc:interceptor>
    <!-- 配置商品被拦截器拦截 -->
    <mvc:mapping path="/item/**" />
    <!-- 配置具体的拦截器 -->
    <bean class="cn.itcast.ssm.interceptor.LoginHandlerInterceptor" />
</mvc:interceptor>
```